

Setup Guide

Hosting, file structure & how the site is built

Who this is for

For whoever sets up or maintains the site at a technical level -- deploying it, finding the right files, or handing the project to an AI assistant. To just update hours or a special, use the Editor Guide instead.

1. Where the site lives

The site runs for free on Cloudflare Pages.

- Live site: <https://sample-business-concept.pages.dev>
- Cloudflare account login: [fill in]

Source files live in the project folder. Only the site/ subfolder gets deployed. The .html files in the folder root are working copies for local preview -- never deployed. Everything else (notes, build scripts, guides) stays private.

Deploy command (PowerShell)

```
npx wrangler pages deploy "site" --project-name sample-business-concept --branch main  
--commit-dirty=true
```

Requires the CLOUDFLARE_API_TOKEN environment variable -- see AI-INSTRUCTIONS.md in the project folder for the token and the full one-line command.

2. Architecture -- how the site is built

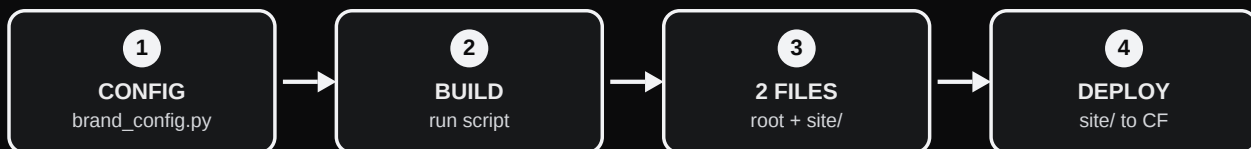
A static HTML site generated from Python scripts. No database, no live backend -- each page is one self-contained .html file with inline CSS and embedded images, produced by running a build script.

Never hand-edit the .html files. Always edit brand_config.py (facts, colors) or the matching build script, then re-run. Hand-edits are silently overwritten.

File map

Build script	Generates
build-scripts/build_homepage.py	root homepage + site/index.html
build-scripts/build_menu_embed.py	root menu + site/menu.html
build-scripts/build_all.py	everything above, in one command

How a build works



- All business facts and colors are in brand_config.py -- one file.
- Each script writes TWO copies: a root working file and the deployable site/ file.
- Images in assets/ are embedded at build time, so pages are self-contained.
- Run build-scripts/build_all.py to rebuild the whole site at once.

Safe editing pattern

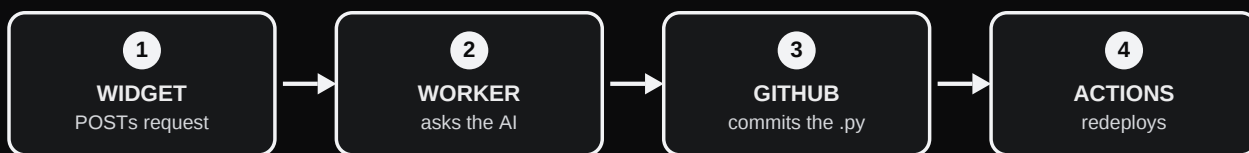
```
old = "HOURS_TEXT = 'Open Daily 8:00am&ndash;9:30pm'"
new = "HOURS_TEXT = 'Open Daily 8:00am&ndash;10:00pm'"
assert old in content
content = content.replace(old, new, 1)
```

3. Built-in behaviors worth knowing

- Back-to-top button: appears bottom-right after scrolling.
- Live menu: the Menu page embeds an external ordering menu -- product names, prices and stock update automatically, no site edits needed.

4. The AI-edit pipeline (Admin Styles)

There's a second layer on top of everything above: a private tool called `admin_styles_sample_business.html` that lets the business owner request edits in plain English (see the Editor Guide) without ever touching build scripts or a terminal. It's a separate local file, not yet part of the site/ deploy folder -- it isn't publicly reachable, by design, until the pipeline below is fully signed off.



Admin Styles' chat POSTs {client, message, code} to one shared Cloudflare Worker (`och-ai-site-editor`) used by every client site. The Worker reads that client's config file (`brand_config.py`) straight from GitHub, asks an AI model to make only the requested edit, sanity-checks that the file structure survived, and commits the result back to GitHub -- which triggers the same GitHub Actions -> Cloudflare Pages deploy you'd otherwise run by hand. Typically live in well under a minute.

Each client is pinned to an AI provider in the Worker's own source (Anthropic by default; NVIDIA or Groq have been used as fallbacks when a provider's endpoint was timing out; a bring-your-own-key Gemini mode also exists as an opt-in per client). The Worker's per-client config (GitHub owner/repo/path, which provider) and its reference source both live in `Workspace.txt` at the project root -- edit that before it's redeployed to Cloudflare.

Secrets never live in a file. Each client's GitHub token, widget access code, and any bring-your-own API key are Worker secrets set directly in the Cloudflare dashboard, not written anywhere in this project. The widget itself only ever asks the owner for their access code, which the Worker checks before making any edit.

The Worker only ever edits the `.py` config file, the same file this guide's build scripts read -- it never touches the built `.html` directly, so the "never hand-edit `.html`, always edit then rebuild" rule above holds no matter which path a change came through.

5. After any change

- Run the script (`python build-scripts/build_all.py`).
- Confirm it prints "written ###" lines with no errors.
- Deploy the site/ folder with the wrangler command on page 1.
- Business facts must match `example.com` -- the official site is the source of truth.
- If the change came through Admin Styles instead, GitHub Actions already redeployed it -- just confirm the live site reflects it.